# TD-TP 6 : À propos de gestion de bâtiments

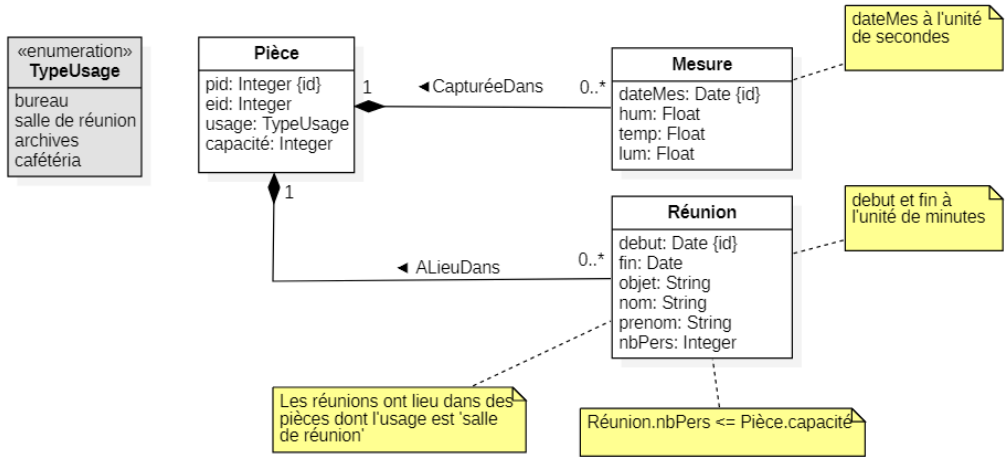
On considère un bâtiment qui comporte de nombreux étages et à chaque étage, plusieurs pièces. On veut mettre en place un système de contrôle de ce bâtiment qui gère des données liées à l'environnement et à l'occupation des différentes pièces.

Chaque pièce, identifiée par un numéro et située à un étage, est équipée d'un capteur de température, d'humidité et de luminosité. Le capteur envoie, à intervalles réguliers (toutes les 2 minutes 30 environ), par un réseau sans fil, ces informations à une base de données. Soit I un instant donné dans l'unité seconde, par exemple '21/05/2006 12h45min42s', pour chaque pièce, les valeurs de température, d'humidité et de luminosité à l'instant I sont issues de la mesure la plus récente inférieure ou égale à I. Notez aussi que l'envoi de mesures a pu éventuellement dysfonctionner. Dans ce cas, on aura des mesures avec leurs time-stamp mais sans valeurs.

Chaque pièce, d'une capacité donnée est dédiée à un usage particulier (salle de réunion, bureau, salle photocopieurs, etc.).

Des réunions sont organisées dans le bâtiment. Pour chacune, on note la pièce où elle aura lieu, ses dates de début et de fin (deux dates à l'unité minute : par exemple '21 février 2007, 11H20'). Sont représentés aussi son objet (comité de direction, séminaire, etc.), le nom et le prénom de la personne qui organise la réunion et le nombre prévu de personnes. Les informations enregistrées dans la base de données peuvent concerner des réunions terminées (dans le passé) ou prévues dans le futur.

**Modèle conceptuel UML :**




**Modèle relationnel :**

LesPièces (pid, eid, usage, capacité)
*/* <p, e, u, c> ∈ LesPièces ⟺ la pièce identifiée par p est située à l'étage e. Son usage est u et sa capacité c. */*

LesMesures (pid, dateMes, hum, temp, lum)
*/* <p, d, h, t, l> ∈ LesMesures ⟺ le capteur de la pièce identifiée par p, a mesuré à la date d (donnée dans l'unité seconde) un pourcentage d'humidité h, une température t et une luminosité de l. */*

LesRéunions (pid, début, fin, objet, nom, prénom, nbPers) */* pid, fin est une autre clef. */*
*/* <s, d, f, o, n, p, nb> ∈ LesRéunions ⟺ une réunion a lieu dans la pièce s, elle se déroule pendant l'intervalle de temps [d, f], son objet est o et nb est le nombre de personnes attendues. La personne de nom n et de prénom p est à l'origine de la demande. d et f sont des dates dans l'unité minute et d < f. */*

Les domaines :

domaine (pid) = domaine (nbPers) = domaine (eid) = entier $> 0$, domaine (capacité) = entier $\geq 0$
domaine (hum) = domaine (temp) = domaine (lum) = réel $\geq 0$,
domaine (usage) = {'salle de réunion', 'bureau', 'cafétéria' ou 'archives'}
domaine (dateMes) = date(seconde) */* date dans l'unité seconde, par exemple, 21/05/2007 12h24m34s */*
domaine (début) = domaine (fin) = date(minute)
*/* date dans l'unité minute, par exemple, 21/05/2007 12h25m */*
domaine (objet) = domaine (nom) = domaine (prénom) = chaîne de caractères

Les contraintes d'intégrité :

- LesMesures[pid] $\subseteq$ LesPièces[pid]
- LesRéunions[pid] $\subseteq$ LesPièces[pid]
- A chaque instant (une date à l'unité minute), une pièce est occupée par 0 ou 1 réunion
- Les réunions ont lieu dans des pièces dont l'usage est 'salle de réunion'
- Le nombre prévu de participants à une réunion est inférieur ou égal à la capacité de la salle choisie pour cette réunion.
- L'attribut eid est obligatoire dans LesPièces

# 1 Consignes

Exprimer dans le langage SQL, les requêtes ci-dessous en respectant les consignes suivantes :

- Les requêtes devront construire des résultats **sans répétition** de valeurs.
- La clause DISTINCT sera utilisée **uniquement lorsque nécessaire**.
- Les **produits de relations** devront être exprimés dans la clause FROM.
- Les règles de **style** seront prises en compte lors de futures évaluations donc entraînez-vous !

**Remarques :**

- Pour comparer des valeurs de type date, on utilisera les opérateurs de comparaison habituels ($<$, $>$, $=$, $<>$, $\leq$, $\geq$). On dispose de plus des fonctions to_date et to_char fournies par Oracle :
  format : {'DD/MM/YYYY', 'DD-Month-YYYY HH24 :MI', 'HH24 :MI :SS', etc. }
  */* Voir la liste des formats supportés par Oracle. */*
  to_date : une chaîne de caractères, un format $\longrightarrow$ une date
  */* to_date (c, f) est la date déduite de la forme externe c, donnée sous forme de chaîne de caractères, selon le format f. */*
  to_char : une date, un format $\longrightarrow$ une chaîne de caractères
  */* to_char (d, f) est la forme externe de la date d selon le format f. */*

# 2 Requêtes

1. Donner les réunions (pid, debut, objet, usage) qui ont lieu dans une salle dont le type n'est pas 'salle de reunion'.

2. Construire les nuplets <p, nb, moy> tels que p est le numéro d'une salle, nb (nb $\geq 1$) est le nombre de réunions planifiées dans p et moy est le nombre moyen de personnes attendues dans les réunions planifiées dans p. On ne considère ici que les salles dans lesquelles au moins une réunion a eu lieu. Ordonner les résultats selon les moyennes (**arrondies au dixième**) de manière descendante.

3. Modifier ensuite la requête pour faire apparaître en plus les pièces dans lesquelles il n'y a jamais eu de réunion (mettre 0 à la place de la valeur NULL de la moyenne).

4. A chaque étage pour lequel on dispose de mesures, donner la pièce ou les pièce(s) ou a été relevée la temperature minimale (**mesurée parmi les pièces de l'etage**). Ordonner les résultats par étage. Schéma attendu : (eid, pid).

5. Ecrire la requête qui retourne les pièces et horaires qui ne vérifient pas la contrainte :

   *A chaque instant (une date à l'unité minute), une pièce est occupée par 0 ou 1 réunion.*

   Principe de la solution : deux réunions sont en conflit si leurs intervalles de temps se chevauchent. Soient I1=[d1, f1] et I2=[d2, f2] deux intervalles de temps tels que f1 $\geq$ d1 et f2 $\geq$ d2, I1 et I2 se chevauchent ssi : I1 $\cap$ I2 $\neq \emptyset$. I1 $\cap$ I2 $= \emptyset \iff$ f1 $<$ d2 ou f2 $<$ d1

   Donc finalement, I1 et I2 se chevauchent ssi : non (f1 $<$ d2 ou f2 $<$ d1). On ajoute aussi la condition d1 $<$ d2 pour ne construire qu'un sens de chevauchement.

   Construire un résultat avec le schéma (pid, jourheuredebutreunion1, jourheurefinreunion1, jourheuredebutreunion2, jourheurefinreunion2) trié par ordre de d1 croissant. Pour les dates, utiliser le format `DD/MM/YYYY HH24:MI`

## 3 Une requête complexe pas à pas

6. Pour chaque réunion, organisée à une date donnée (commencer par l'exemple du 5 décembre 2021 avant de paramètrer la requête par n'importe quelle date), donner la pièce où elle est planifiée, ses instants de début et de fin, les valeurs de températures et l'instant où elles ont été mesurées pendant le déroulement de la réunion (c'est-à-dire toutes les valeurs de températures et instants de mesures effectuées entre l'instant de début et l'instant de fin de la réunion).

   **Attention** : les granularités des dates sont plus fines que le jour. Pour fixer une date à la granularité du jour, il faut utiliser : `to_char(debut,'DD/MM/YYYY')='05/12/2021'`, et pas dans l'autre sens avec `to_date`.

   **Conseil** : ajuster le nombre de lignes affichables avant de répéter les entêtes, ajuster le nombre de caractères par ligne. Formater éventuellement la taille des différents attributs. Afficher toutes les dates à la granularité de la seconde (`DD/MM/YYYY HH24:MI:SS`)

7. On souhaite faire apparaitre plus clairement les valeurs nulles de température, correspondant au cas où le capteur n'a pas fonctionné. Remplacer les valeurs nulles par une suite de `'*'`.

   Modifier la requête précédente pour faire cet ajout. **Indication** : attention '*****' est une chaine de caractères, qu'on veut placer dans la même colonne que temp qui est un réel. Ce n'est pas possible d'avoir des domaines différents dans une même colonne, par contre, on peut utiliser `to_char(temp)` pour transformer une température réelle en chaine de caractère

8. On souhaite ajouter le cas des salles pour lesquelles il n'y a eu aucune température mesurée entre le début et la fin de la réunion.

   Modifier la requête précédente pour prendre en compte ce cas.

9. Les valeurs de l'intervalle [début..fin] étant des valeurs mesurées, on souhaite ajouter, dans l'affichage du résultat, à la suite de la température, la chaîne de caractères `'(m)'` (pour mesuré).

   Modifier la requête précédente pour faire cet ajout.

   **Indication** : on pourra utiliser l'opérateur `||` qui concatène deux chaines de caractères. Exemple : `'a' || 'b'` construit `'ab'`

10. On veut maintenant ajouter les valeurs des mesures à la frontière de l'intervalle [debut..fin], notamment pour le cas où il n'y a pas eu de mesure à l'instant exact début et/ou à l'instant exact fin et/ou le cas où aucune valeur n'a été mesurée entre début et fin.

    On choisit d'ajouter, à l'affichage, les températures mesurées les plus proches de l'intervalle suffixées par `'(i)'`, c'est-à-dire pour chaque réunion :
    — la température la plus proche de début parmi toutes celles inférieures à début
    — la température la plus proche de fin parmi toutes celles supérieures à fin
    Modifier la requête précédente pour faire cet ajout.